Module 04 — Automation

FOR DUMMIES

Goal: replace module 03's hand-typing with one idempotent script, understand the template + data model well enough to modify it, and write your first bespoke Nornir script from scratch.

If you took Aston's original automation course, this is the same Netmiko → Nornir → NAPALM arc — but pointed at the fabric you just built instead of GNS3 behind a jump box, and with zero inventory or credential setup, because Containerlab generates all of that on every deploy.

---

1. Link the inventory (30 seconds, easy to forget)

Containerlab wrote a fresh Nornir inventory when you deployed the fabric — management IPs, default credentials, all of it. Point the automation at it:

```
cd ~/clab-bootstrap
./automation/link-inventory.sh ~/clab-bootstrap/topologies/clab-spineleaf
# (or wherever clab-spineleaf landed — the deploy output printed it)
netauto
cd automation/nornir
```

This is the step from module 00's decision tree that follows *every* topology deploy. Forget it and your scripts talk to the previous lab's IPs, with confusing results.

2. The ladder, in twenty minutes

Run scripts 01–03 in order against the fabric. Each is one rung; read each script's docstring — they're short — before running it.

```
python3 01_netmiko_hello.py     # rung 1: one SSH session, one command, print
python3 02_nornir_scale.py      # rung 2: same task, ALL FOUR nodes, concurrent
python3 03_napalm_facts.py      # rung 3: structured JSON instead of scraped text
```

What to actually notice at each rung:

- **01 → 02:** the task logic didn't change; the *scale* did. Nornir owns the inventory and the threading, so four devices or four hundred is the same script. This is why frameworks exist.
- **02 → 03:** compare the outputs. 02 gave you text you'd have to regex (fragile — one EOS wording change breaks it). 03 gives you dicts with keys. Structured data is what makes the next rung *possible*: you can't diff prose, but you can diff data.

3. Rung 4: the underlay, declaratively

Module 03's checkpoint left the fabric hand-configured and saved. Now:

```
python3 05_spineleaf_underlay.py
```

Read the output carefully. For any node where your hand-typed config already matches the desired state exactly: `changed=False`, nothing sent. Where you deviated — a missing description, a typo'd mask — you'll see a **diff of exactly the delta**, and only that delta was pushed. Then:

```
python3 05_spineleaf_underlay.py     # again
```

All nodes: `changed=False`. Run it fifty times; still nothing. This is **idempotency** — the script describes a *goal*, not a list of steps — and it's the single concept that separates real network automation from "a script that types fast." A fast-typing script run twice configures everything twice. This one run twice does nothing, safely.

4. Open the hood: template + data

Two files produced everything you just watched:

The template — `templates/spineleaf_underlay.j2` — is module 03's config with the values replaced by holes:

```
interface {{ intf.name }}
   ip address {{ intf.ip }}
```

The data — the `FABRIC` dict in `05_spineleaf_underlay.py` — is module 03's IP plan table, as Python. Loopbacks, router-ids, interface lists per node.

The script is just the plumbing between them: render template with host's data → hand result to NAPALM → NAPALM diffs against the device → push the delta if any. That separation is the entire model:

Change the config's *shape* (new OSPF timer on every node) → edit the **template**. **Change a *value*** (leaf2's loopback) → edit the **data**. Either way → run the script → devices converge on the goal.

Prove it to yourself — both directions:

1. **Data change:** in `FABRIC`, edit leaf2's Ethernet1 description. Run the script: one node changes, three say `changed=False`.
2. **Template change:** add `ip ospf cost 100` under the interface block in the template. Run: all four nodes change (every fabric interface got it). Then delete the line and run again — and notice the cost does **not** get removed. Merge-mode pushes what's in the template; it doesn't remove what isn't. Clean it up by hand (`no ip ospf cost` on the interfaces, or just note it). That asymmetry — merge vs. full config replace — is a real design decision in production automation, and now you've met it.

5. Write your own: `verify_fabric.py`

The scripts so far *push* config. Just as valuable: scripts that *check* state. Write a bespoke one — a fabric health check that prints, per device, its hostname, EOS version, and OSPF neighbor count, and flags any device with fewer than 2 neighbors.

Skeleton to start from (this is `02` + `03`'s patterns combined):

```python
#!/usr/bin/env python3
"""Fabric health check — my first bespoke Nornir script."""
from nornir import InitNornir
from nornir_napalm.plugins.tasks import napalm_get
from nornir_netmiko.tasks import netmiko_send_command
from creds import apply_credential_override

EXPECTED_NEIGHBORS = 2

def check(task):
    facts = task.run(task=napalm_get, getters=["facts"])
    neigh = task.run(task=netmiko_send_command,
                     command_string="show ip ospf neighbor | include FULL")
    count = len([l for l in neigh.result.splitlines() if l.strip()])
    status = "OK " if count >= EXPECTED_NEIGHBORS else "FAIL"
    print(f"[{status}] {task.host.name}: "
          f"EOS {facts.result['facts']['os_version']}, "
          f"{count} OSPF neighbors")

def main():
    nr = InitNornir(config_file="config.yaml")
    apply_credential_override(nr)
    nr.run(task=check)

if __name__ == "__main__":
    main()
```

Get it running, then make it yours — ideas in rough difficulty order: also verify loopback-to-loopback pings (`ping ... source ...` via netmiko); exit nonzero if anything FAILs (now it's CI-able); check for interfaces that are up but missing a description (a real audit people get paid for). Break the fabric (`shutdown` an uplink) and confirm your script catches it — a health check you've never seen fail isn't tested.

6. Checkpoint

`05_spineleaf_underlay.py` twice in a row: all `changed=False`

You changed a value via **data** and a behavior via **template**, predicted the blast radius of each before running, and were right

You can say aloud what NAPALM adds over Netmiko (diff + merge against structured config, enabling idempotency)

Your `verify_fabric.py` runs, and catches a real failure

What you learned

The framework ladder and why each rung exists; idempotency as *the* automation concept; the template/data separation and merge-mode's sneaky asymmetry; and that writing a bespoke Nornir script is ~30 lines once the inventory problem is someone else's. Which, here, it permanently is — Containerlab regenerates it every deploy.

Next: Module 05 — VXLAN + EVPN